📝 GLA UNIVERSITY — DAY 1 POST-ASSESSMENT QUESTION PAPER

High Performance Computing & Heterogeneous AI Systems (`GLA-HPC-BCSG701`)

Time Allowed: 60 Minutes | **Total Questions:** 50 Multi-Choice Questions | **Maximum Marks:** 50
Candidate Name: _____ | **Roll Number:** _____
Date: _____ | **Section:** _____

Instructions to Candidates:

1. Each question carries 1 mark. There is no negative marking.
2. Select only ONE option (a, b, c, or d) for each question.
3. Carefully examine all mathematical derivations, code snippets, and shell commands before marking your answer.

Question 01 [Module 1: HPC Principles & Architectures]

Question: An HPC dual-socket compute node contains two Intel Xeon Platinum 8480+ processors. Each processor has 56 cores running at a base vector frequency of 2.0 GHz. Each core has two 512-bit FMA (Fused Multiply-Add) execution units for double precision (FP64). What is the theoretical peak double-precision performance of this dual-socket node?

- **a)** 3.58 TFLOPS
- **b)** 7.168 TFLOPS
- **c)** 14.336 TFLOPS
- **d)** 28.672 TFLOPS

Candidate Selection: [ ]

Question 02 [Module 1: HPC Principles & Architectures]

Question: A molecular dynamics simulation code spends 15% of its total execution time in sequential file I/O and initialization that cannot be parallelized. What is the maximum theoretical speedup of this application when running on a 64-core cluster node according to Amdahl's Law?

- **a)** 6.12x
- **b)** 10.0x
- **c)** 5.77x
- **d)** 64.0x

Candidate Selection: [ ]

Question 03 [Module 1: HPC Principles & Architectures]

Question: Why does Gustafson's Law offer a more realistic speedup model than Amdahl's Law for modern large-scale supercomputing clusters running scientific simulations?

- **a)** Gustafson's Law assumes the number of processor cores remains constant while clock speed increases
- **b)** Gustafson's Law models weak scaling, where the problem size increases proportionally with the number of compute processors
- **c)** Gustafson's Law proves that sequential fractions can be executed in zero time on quantum processors
- **d)** Gustafson's Law only applies to GPUs, whereas Amdahl's Law only applies to CPUs

Candidate Selection: [ ]

Question 04 [Module 1: HPC Principles & Architectures]

Question: Under the Roofline Performance Model, an algorithm has an arithmetic intensity of 0.25 FLOPs/byte. The target HPC compute node has a peak memory bandwidth of 400 GB/s and a peak compute performance of 8.0 TFLOPS. What is the maximum performance attainable by this algorithm, and what is its limiting bound?

- **a)** 100 GFLOPS, Memory Bandwidth Bound
- **b)** 2.0 TFLOPS, Compute Bound
- **c)** 8.0 TFLOPS, Compute Bound
- **d)** 1.6 TFLOPS, Latency Bound

Candidate Selection: [ ]

Question 05 [Module 1: HPC Principles & Architectures]

Question: What is the defining characteristic of a 'Full Bisection Bandwidth' (non-blocking) Fat-Tree interconnect in an enterprise HPC cluster?

- **a)** All nodes in the cluster share a single physical copper ethernet cable
- **b)** Any arbitrary half of the compute nodes can communicate simultaneously with the other half at full network line rate without oversubscription
- **c)** The network switches turn off automatically when compute nodes are idle
- **d)** Communication latency increases by a factor of 10 for every additional rack added

Candidate Selection: [ ]

Question 06 [Module 1: HPC Principles & Architectures]

Question: Why is InfiniBand with RDMA (Remote Direct Memory Access) preferred over standard TCP/IP Ethernet for distributed HPC applications like MPI?

- **a)** InfiniBand uses less electricity because it operates without network switches
- **b)** RDMA allows network adapters to transfer data directly between application memories across nodes without CPU OS kernel context switches and copying
- **c)** TCP/IP cannot transmit floating-point data across physical networks
- **d)** InfiniBand cables only support wireless radio transmission

Candidate Selection: [ ]

Question 07 [Module 1: HPC Principles & Architectures]

Question: In a modern heterogeneous HPC supercomputing node (CPU + GPU/Accelerator), what is the primary division of architectural responsibility?

- **a)** The GPU handles file I/O and OS interrupts, while the CPU performs deep learning matrix multiplication
- **b)** The CPU acts as the latency-optimized orchestrator/control plane, while the GPU acts as the high-throughput parallel data execution engine
- **c)** The CPU and GPU must run completely identical instruction sets at the exact same clock speed
- **d)** The GPU replaces the system motherboard and BIOS entirely

Candidate Selection: [ ]

Question 08 [Module 1: HPC Principles & Architectures]

Question: How does a distributed parallel file system (such as Lustre or BeeGFS) prevent I/O bottlenecks when thousands of cluster compute cores write checkpoints simultaneously?

- **a)** By compressing all files into a single ZIP archive on the head node
- **b)** By separating metadata operations (MDT) from data storage (OST) and striping file chunks across multiple physical storage servers concurrently
- **c)** By disabling file write permissions and running purely in RAM
- **d)** By requiring each compute node to write to an individual USB flash drive

Candidate Selection: [ ]

Question 09 [Module 2: Intel HPC Architecture & Hardware Accelerators]

Question: In Intel 4th and 5th Gen Xeon Scalable processors (Sapphire Rapids / Emerald Rapids), what are the two core hardware components of Intel AMX?

- **a)** L1 Data Cache and Translation Lookaside Buffer (TLB)
- **b)** Eight 1KB 2D Tile Registers (TMM0-TMM7) and the Tile Matrix Multiply Unit (TMUL)
- **c)** PCIe Gen 5 bus controller and CXL memory expander
- **d)** AVX-256 integer ALU and IEEE-754 floating-point divider

Candidate Selection: [ ]

Question 10 [Module 2: Intel HPC Architecture & Hardware Accelerators]

Question: How do AVX-512 opmask registers (k0 through k7) improve performance when vectorizing conditional loops (`if-else` branches) in HPC scientific code?

- **a)** They automatically increase the CPU core clock frequency during `if` statements
- **b)** They allow conditional vector operations via bitmask predication, executing active vector lanes while suppressing inactive lanes without branch misprediction penalties
- **c)** They convert 32-bit floats into 64-bit doubles automatically
- **d)** They disable cache coherency across cores during conditional checks

Candidate Selection: [ ]

Question 11 [Module 2: Intel HPC Architecture & Hardware Accelerators]

Question: What is the primary architectural benefit of configuring Sub-NUMA Clustering (SNC-2 or SNC-4) in the BIOS of a dual-socket Intel Xeon server for HPC workloads?

- **a)** It disables DDR5 ECC checking to achieve higher memory clock rates
- **b)** It partitions the physical processor and its memory controllers into smaller localized NUMA domains, reducing average L3 cache and memory access latency
- **c)** It forces all MPI ranks to communicate using Ethernet instead of shared memory
- **d)** It allows Windows and Linux to boot simultaneously on separate cores

Candidate Selection: [ ]

Question 12 [Module 2: Intel HPC Architecture & Hardware Accelerators]

Question: When an HPC node executes heavy AVX-512 512-bit FMA workloads, why might the Linux `turbostat` utility report a core frequency slightly lower than the scalar base frequency?

- **a)** The Linux kernel has encountered a hardware fault in the CPU voltage regulator
- **b)** Intel Xeon processors switch to an optimized AVX-512 power license level with dynamic voltage/frequency scaling to prevent exceeding thermal and electrical design current (TDP/ICCMax) limits
- **c)** AVX-512 instructions disable hyperthreading permanently
- **d)** The system RAM runs out of bandwidth and turns off CPU cores

Candidate Selection: [ ]

Question 13 [Module 2: Intel HPC Architecture & Hardware Accelerators]

Question: Which advanced packaging technology and memory architecture are utilized in the Intel Data Center GPU Max Series (formerly code-named Ponte Vecchio) to deliver massive HPC throughput?

- **a)** Single monolithic silicon die with GDDR6 memory
- **b)** EMIB (Embedded Multi-die Interconnect Bridge) and Foveros 3D die stacking with High Bandwidth Memory (HBM2e)
- **c)** Wire-bonded packaging with standard DDR4 SODIMM sockets
- **d)** Optane Persistent Memory connected via SATA interface

Candidate Selection: [ ]

Question 14 [Module 2: Intel HPC Architecture & Hardware Accelerators]

Question: What unique networking architectural design distinguishes Intel Gaudi 2 / Gaudi 3 AI processors from traditional GPU clusters?

- **a)** They do not require power cables and run on solar energy
- **b)** They integrate multiple 100GbE / 200GbE RoCEv2 (RDMA over Converged Ethernet) ports directly on the silicon accelerator package, eliminating external PCIe NICs

- **c)** They only communicate over USB 3.0 serial connections
- **d)** They prohibit multi-node distributed training across racks

Candidate Selection: [ ]

Question 15 [Module 2: Intel HPC Architecture & Hardware Accelerators]

Question: You run `lscpu | grep -i flags` on an HPC cluster node and see the flags `avx512f`, `amx_tile`, and `amx_bf16`. What does this indicate about the host CPU?

- **a)** It is an older AMD Opteron processor from 2012
- **b)** It is an Intel Xeon Scalable processor (Sapphire Rapids or newer) with hardware support for AVX-512 and matrix multiplication acceleration for BF16 precision
- **c)** It is an ARM Cortex-A53 processor without vector extensions
- **d)** The system is running on a software emulation layer without hardware acceleration

Candidate Selection: [ ]

Question 16 [Module 3: Linux Operating System Foundations for HPC]

Question: In the Linux filesystem architecture, what happens at the metadata level when an engineer creates a hard link to an existing file using `ln fileA linkB`?

- **a)** A new inode is created with a duplicate copy of the data blocks
- **b)** A new directory entry `linkB` is created pointing to the identical inode number as `fileA`, and the inode's link count is incremented by 1
- **c)** The original file `fileA` is deleted immediately
- **d)** The file is converted into a Windows shortcut file

Candidate Selection: [ ]

Question 17 [Module 3: Linux Operating System Foundations for HPC]

Question: An HPC administrator sets `umask 027` in `/etc/profile`. What will be the exact default permission strings and octal modes for newly created directories and regular files respectively?

- **a)** Directories: `rwxr-x---` (750); Regular Files: `rw-r-----` (640)
- **b)** Directories: `rwxrwxrwx` (777); Regular Files: `rw-rw-rw-` (666)
- **c)** Directories: `rwxr-xr-x` (755); Regular Files: `rw-r--r--` (644)
- **d)** Directories: `rwx------` (700); Regular Files: `rw-------` (600)

Candidate Selection: [ ]

Question 18 [Module 3: Linux Operating System Foundations for HPC]

Question: Why does an HPC system administrator apply the SGID bit to a collaborative project directory via `chmod 2775 /shared/research_data`?

- **a)** To allow any student to delete other students' files without permission

- **b)** To ensure all new files and subdirectories created inside `/shared/research_data` automatically inherit the group ownership of the parent directory rather than the creator's primary group
- **c)** To encrypt the directory with AES-256 automatically
- **d)** To force the directory to execute as root

Candidate Selection: [ ]

Question 19 [Module 3: Linux Operating System Foundations for HPC]

Question: In an HPC simulation pipeline, what is the precise technical difference between a 'Zombie' process and an 'Orphan' process in Linux?

- **a)** A Zombie process consumes 100% CPU; an Orphan process consumes 100% RAM
- **b)** A Zombie process has completed execution but its parent has not yet read its exit status via `waitpid()`; an Orphan process is still running after its parent has terminated and is adopted by PID 1 (systemd)
- **c)** Zombie processes only exist on Windows; Orphan processes only exist on Linux
- **d)** An Orphan process has been killed by the OOM killer; a Zombie process is waiting for network packets

Candidate Selection: [ ]

Question 20 [Module 3: Linux Operating System Foundations for HPC]

Question: Where in the Linux virtual filesystem can an HPC researcher programmatically inspect real-time kernel memory statistics, active CPU topology, and hardware interrupts without invoking binary tools?

- **a)** Under the `/etc/sysconfig` folder
- **b)** Under the `/proc` virtual pseudo-filesystem (e.g., `/proc/meminfo`, `/proc/cpuinfo`, `/proc/interrupts`)
- **c)** Inside the `/tmp` temporary swap partition
- **d)** Within the `/usr/share/man` documentation directories

Candidate Selection: [ ]

Question 21 [Module 3: Linux Operating System Foundations for HPC]

Question: When running `free -h` on an HPC server, the `used` memory is 12 GB, but `buff/cache` is 500 GB out of 512 GB total RAM. Why should the administrator NOT panic?

- **a)** The RAM is defective and must be physically replaced
- **b)** Linux aggressively uses free RAM as Page Cache to accelerate disk I/O; this cached memory is immediately reclaimable if compute applications request memory
- **c)** A memory leak virus has infected the Linux kernel
- **d)** The system has swapped all data to the local hard drive

Candidate Selection: [ ]

Question 22 [Module 3: Linux Operating System Foundations for HPC]

Question: An engineer compiles an Intel oneAPI application, but running `./vector_add` fails with: `error while loading shared libraries: libmkl_intel_lp64.so.2: cannot open shared object file`. What is the root cause and remedy?

- **a)** The file system is corrupted and must be reformatted
- **b)** The dynamic linker (`ld.so`) cannot locate oneAPI runtime libraries; the remedy is to set `LD_LIBRARY_PATH` by running `source /opt/intel/oneapi/setvars.sh`
- **c)** The CPU does not support 64-bit computation
- **d)** The binary was compiled for ARM architecture instead of x86_64

Candidate Selection: [ ]

Question 23 [Module 4: Process Management, Signals & Resource Limits]

Question: Why should an HPC cluster administrator always attempt to terminate an errant job with `kill -15 <pid>` (SIGTERM) before resorting to `kill -9 <pid>` (SIGKILL)?

- **a)** SIGTERM executes 10 times faster than SIGKILL
- **b)** SIGTERM can be caught by the application's signal handler to flush buffers, save scientific checkpoints, and close open network sockets cleanly; SIGKILL immediately destroys the process table entry without cleanup
- **c)** SIGKILL requires root permissions while SIGTERM does not
- **d)** SIGTERM only pauses the process for 5 seconds

Candidate Selection: [ ]

Question 24 [Module 4: Process Management, Signals & Resource Limits]

Question: An urgent high-priority interactive debug job arrives on a shared node. How can an administrator temporarily suspend a running background simulation without terminating it or losing its in-memory calculation state, and resume it later?

- **a)** `kill -STOP <pid>` to pause execution; `kill -CONT <pid>` to resume execution
- **b)** `kill -9 <pid>` to pause; `kill -1 <pid>` to resume
- **c)** `nice -n 20 <pid>` to pause; `renice -n 0 <pid>` to resume
- **d)** `chmod 000 <binary>` to pause; `chmod 755 <binary>` to resume

Candidate Selection: [ ]

Question 25 [Module 4: Process Management, Signals & Resource Limits]

Question: When a node exhausts physical RAM and swap, how does the Linux OOM Killer decide which process to kill, and how can a critical cluster manager daemon (`slurmd`) be protected?

- **a)** It kills processes randomly; daemons cannot be protected
- **b)** It calculates a heuristic score based on RAM usage and runtime (`/proc/[pid]/oom_score`); daemons can be protected by writing `-1000` to `/proc/[pid]/oom_score_adj`
- **c)** It always kills the process with PID 1 first
- **d)** It deletes disk files until memory becomes free

Candidate Selection: [ ]

Question 26 [Module 4: Process Management, Signals & Resource Limits]

Question: Why is binding an HPC simulation thread to specific physical CPU cores using `taskset -c 0-7` or `numactl --physcpubind=0-7 --membind=0` critical for achieving reproducible benchmarks?

- **a)** It prevents the operating system from updating the system clock
- **b)** It eliminates cache thrashing and inter-socket NUMA latency caused by the OS scheduler randomly migrating threads between different physical sockets
- **c)** It doubles the PCIe bus bandwidth automatically
- **d)** It bypasses CPU thermal sensors to prevent throttling

Candidate Selection: [ ]

Question 27 [Module 4: Process Management, Signals & Resource Limits]

Question: A student launches a CPU-intensive data pre-processing script on a shared login headnode using `nice -n 19 python preprocess.py`. What does this niceness value dictate to the Linux scheduler?

- **a)** It assigns the highest possible real-time priority, preempting all other users
- **b)** It sets the lowest possible priority, causing the process to yield CPU cycles whenever any normal or higher-priority job requires computing time
- **c)** It limits the script to run only on CPU 19
- **d)** It allocates 19 GB of dedicated RAM to the Python process

Candidate Selection: [ ]

Question 28 [Module 4: Process Management, Signals & Resource Limits]

Question: Why do HPC administrators explicitly set `ulimit -s unlimited` and `ulimit -l unlimited` in cluster batch environments before running OpenMP or MPI jobs?

- **a)** To allow unlimited network internet downloads
- **b)** To prevent segmentation faults caused by large thread-private stack allocations and enable InfiniBand drivers to lock physical memory pages (pinning) for RDMA transfers
- **c)** To allow users to create an infinite number of user accounts
- **d)** To permanently disable the Linux firewall

Candidate Selection: [ ]

Question 29 [Module 4: Process Management, Signals & Resource Limits]

Question: An HPC compute node with 64 physical CPU cores reports a 1-minute load average of 80.0. What does this metric signify regarding system load?

- **a)** 80% of the RAM has been consumed by the operating system
- **b)** All 64 cores are fully occupied, and an average of 16 process threads are queued waiting for CPU or stuck in uninterruptible disk I/O (D state)
- **c)** The CPU temperature has reached 80 degrees Celsius
- **d)** The system network interface is transmitting at 80 Gbps

Candidate Selection: [ ]

Question 30 [Module 5: Secure Remote Access & Cluster Networking]

Question: When configuring SSH key-based authentication for an HPC cluster, where must the user's Public Key and Private Key reside respectively?

- **a)** Both the public and private keys must be copied to the remote server's `/tmp` directory
- **b)** The Private Key must remain strictly confidential on the local client machine; the Public Key is appended to `~/.ssh/authorized_keys` on the remote server
- **c)** The Private Key is sent to the remote server, and the Public Key is deleted immediately
- **d)** Neither key is saved; SSH generates a new password every 10 seconds

Candidate Selection: [ ]

Question 31 [Module 5: Secure Remote Access & Cluster Networking]

Question: A student configures passwordless SSH, but the remote cluster repeatedly rejects the key and prompts for a password. Inspecting the remote directory reveals `drwxrwxrwx` (777) on `~/.ssh`. Why does SSH fail?

- **a)** The SSH daemon requires permissions 777 to read the keys
- **b)** OpenSSH enforces strict permission security checks; if `~/.ssh` is group- or world-writable, the SSH daemon considers it compromised and refuses key authentication
- **c)** The student must re-install the Linux operating system
- **d)** OpenSSH does not work on directories with odd octal numbers

Candidate Selection: [ ]

Question 32 [Module 5: Secure Remote Access & Cluster Networking]

Question: Which standard Linux utility automatically creates `~/.ssh` on the remote server with correct permissions (700) and appends the local public key into `~/.ssh/authorized_keys` (600) in a single command?

- **a)** `ssh-copy-id -i ~/.ssh/id_ed25519.pub user@headnode`
- **b)** `scp ~/.ssh/id_ed25519 user@headnode:/etc/passwd`
- **c)** `ssh-keygen -delete user@headnode`
- **d)** `rsync -r ~/.ssh user@headnode:/root`

Candidate Selection: [ ]

Question 33 [Module 5: Secure Remote Access & Cluster Networking]

Question: An HPC dataset transfer of 500 GB was interrupted halfway through due to a temporary network drop. Why is `rsync -avzP` dramatically superior to `scp` for resuming the transfer?

- **a)** `scp` compresses data using quantum algorithms, whereas `rsync` cannot
- **b)** `rsync` utilizes a rolling checksum delta-transfer algorithm with `--partial` to resume exactly where it was interrupted, skipping identical files and transferring only changed byte blocks
- **c)** `rsync` deletes the files on the remote server to save disk space

- **d)** `scp` requires a physical tape drive to resume transfers

Candidate Selection: [ ]

Question 34 [Module 5: Secure Remote Access & Cluster Networking]

Question: In a secure HPC cluster with private compute nodes accessible only through a Bastion / Login node, how does `ssh -A user@login-node` (Agent Forwarding) provide seamless multi-hop authentication without security compromise?

- **a)** It copies the user's private key into `/tmp` on the login node so compute nodes can read it
- **b)** It forwards the local `ssh-agent` authentication socket, allowing the login node to request signature confirmations from the local client machine without exposing the private key to the intermediary node
- **c)** It disables password authentication for all users across the university
- **d)** It converts SSH into an unencrypted telnet connection

Candidate Selection: [ ]

Question 35 [Module 5: Secure Remote Access & Cluster Networking]

Question: Why do high-throughput HPC storage networks and cluster fabrics configure network interfaces with an MTU (Maximum Transmission Unit) of 9000 bytes (Jumbo Frames) instead of the standard 1500 bytes?

- **a)** To allow network cables to stretch 6 times longer
- **b)** To reduce packet fragmentation, lower packet header overhead, and significantly reduce host CPU interrupt processing rates when streaming large datasets
- **c)** Because 9000 bytes is the minimum packet size required by IPv6
- **d)** To prevent unauthorized Wi-Fi connections to the cluster

Candidate Selection: [ ]

Question 36 [Module 5: Secure Remote Access & Cluster Networking]

Question: A researcher runs a Jupyter Lab notebook on remote compute node port 8888. The compute node has no public IP. What SSH tunneling command must be executed on the researcher's local laptop to view the notebook at `http://localhost:8888`?

- **a)** `ssh -R 8888:localhost:8888 user@headnode`
- **b)** `ssh -L 8888:localhost:8888 user@headnode`
- **c)** `ssh -D 1080 user@headnode`
- **d)** `ssh -X user@headnode jupyter-lab`

Candidate Selection: [ ]

Question 37 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation]

Question: Why is it an industry standard to begin all production HPC automation scripts with `set -euo pipefail`?

- **a)** It enables multi-threading and accelerates bash script execution by 400%

- **b)** It ensures the script exits immediately if any command returns a non-zero exit status (`-e`), terminates on uninitialized variable usage (`-u`), and causes pipelines to fail if any sub-command fails (`-o pipefail`)
- **c)** It forces bash to run scripts inside an isolated Docker container
- **d)** It translates the bash script into compiled C++ code automatically

Candidate Selection: [ ]

Question 38 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation]

Question: An HPC simulation creates a 50 GB temporary scratch directory `TMP_DIR=$(mktemp -d /tmp/sim_XXXXXX)`. How can the script guarantee this directory is deleted even if the job is aborted by Ctrl+C or a Slurm preemption signal?

- **a)** By adding `rm -rf $TMP_DIR` as the very last line of the script
- **b)** By registering an exit trap: `trap 'rm -rf "$TMP_DIR"' EXIT INT TERM`
- **c)** By running `chmod 000 $TMP_DIR` before running calculations
- **d)** The Linux kernel automatically deletes all folders named `TMP_DIR` every 10 minutes

Candidate Selection: [ ]

Question 39 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation]

Question: In an automated HPC batch loop, `FILE="/scratch/users/shivam/sim_data_v2.tar.gz"`. How can you extract the filename without its directory path using pure Bash parameter expansion without spawning slow subshells (`basename` / `sed`)?

- **a)** `echo ${FILE%%/*}`
- **b)** `echo ${FILE##*/}`
- **c)** `echo ${FILE%/*}`
- **d)** `echo ${FILE//path/}`

Candidate Selection: [ ]

Question 40 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation]

Question: You have an HPC benchmark log file where column 3 contains runtime in seconds. Which `awk` command accurately computes the arithmetic mean (average) runtime across all non-header lines?

- **a)** `awk '{sum += $3} END {print sum/NR}' benchmark.log`
- **b)** `awk 'print avg($3)' benchmark.log`
- **c)** `awk 'SUM($3)/COUNT' benchmark.log`
- **d)** `awk '{print $3 + $3}' benchmark.log`

Candidate Selection: [ ]

Question 41 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation]

Question: What is the critical behavioral difference between `cmd > log.txt 2>&1` and `cmd 2>&1 > log.txt` in Linux bash?

- **a)** Both commands perform the exact same redirection
- **b)** `cmd > log.txt 2>&1` redirects both stdout and stderr to `log.txt`; `cmd 2>&1 > log.txt` redirects stderr to the terminal screen and only stdout to `log.txt`
- **c)** `cmd 2>&1 > log.txt` creates two separate files
- **d)** `cmd > log.txt 2>&1` is invalid syntax in modern Linux

Candidate Selection: [ ]

Question 42 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation]

Question: How do you declare and populate an associative array (hash map / key-value dictionary) in Bash 4+ to track GPU counts per cluster node?

- **a)** `set -A GPU_MAP; GPU_MAP[node01]=4`
- **b)** `declare -A GPU_MAP; GPU_MAP["node01"]=4; GPU_MAP["node02"]=8`
- **c)** `GPU_MAP = {node01: 4, node02: 8}`
- **d)** `array GPU_MAP = (node01=4, node02=8)`

Candidate Selection: [ ]

Question 43 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation]

Question: An engineer writes a script to dynamically generate a Slurm batch script. Why is using `cat << 'EOF' > job.sh` (quoted delimiter) fundamentally different from `cat << EOF > job.sh` (unquoted delimiter)?

- **a)** The quoted delimiter executes the script in python mode
- **b)** Quoting `'EOF'` prevents the current shell from expanding variables (e.g., `$SLURM_JOB_ID`), preserving variable literals to be evaluated later on the compute node
- **c)** Unquoted `EOF` permanently encrypts the generated file
- **d)** Quoting `'EOF'` limits the file size to 10 lines

Candidate Selection: [ ]

Question 44 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations]

Question: What primary industry challenge does the Intel oneAPI open programming specification directly solve for high-performance computing?

- **a)** It forces all computers worldwide to run the same operating system
- **b)** It eliminates proprietary vendor-locked programming models (e.g., CUDA) by delivering a unified, standards-based C++/SYCL programming model across heterogeneous CPUs, GPUs, FPGAs, and AI accelerators
- **c)** It replaces standard C++ with a new proprietary proprietary binary format
- **d)** It allows computer hardware to operate without compilers

Candidate Selection: [ ]

Question 45 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations]

Question: What is the key functional distinction between the Intel oneAPI Base Toolkit and the Intel oneAPI HPC Toolkit?

- **a)** The Base Toolkit is free, while the HPC Toolkit is only for video games
- **b)** The Base Toolkit contains core developer tools (DPC++/C++ compiler, oneMKL, oneDNN, VTune Profiler); the HPC Toolkit adds distributed cluster tools (Intel Fortran, Intel MPI Library, Cluster Checker)
- **c)** The Base Toolkit only works on Windows; the HPC Toolkit only works on macOS
- **d)** The Base Toolkit compiles code; the HPC Toolkit only monitors hardware temperature

Candidate Selection: [ ]

Question 46 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations]

Question: Why must an engineer execute `source /opt/intel/oneapi/setvars.sh` in their shell before invoking oneAPI compilers (`icpx` , `ifx`) or tools?

- **a)** To download the latest compiler updates from the internet
- **b)** To initialize mandatory environment variables (`ONEAPI_ROOT` , `PATH` , `CPATH` , `LD_LIBRARY_PATH` , `CMAKE_PREFIX_PATH`) pointing to active toolkit installations
- **c)** To reboot the Linux kernel into HPC mode
- **d)** To delete old temporary files in `/tmp`

Candidate Selection: [ ]

Question 47 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations]

Question: In SYCL and DPC++ programming, what does the 'Single-Source' programming paradigm mean?

- **a)** All source code across an entire university must be written in a single file
- **b)** Both host execution code (orchestration, I/O) and device execution kernels (vector arithmetic) are written together within the same C++ file and compiled by a unified compiler toolchain
- **c)** The code can only read data from one single input file
- **d)** Only one developer can edit the code repository at a time

Candidate Selection: [ ]

Question 48 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations]

Question: Examine the following C++ SYCL code snippet: `sycl::queue q(sycl::gpu_selector_v);` . What happens at runtime if this program is executed on an HPC server that has no GPU installed?

- **a)** The computer automatically orders a GPU from the internet
- **b)** The SYCL runtime throws a `sycl::exception` indicating no matching device was discovered
- **c)** The kernel executes silently on the CPU without notifying the user
- **d)** The operating system crashes and reboots

Candidate Selection: [ ]

Question 49 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations]

Question: What is the key technical difference between `sycl::malloc_shared()` and `sycl::malloc_device()` when managing memory in a heterogeneous SYCL application?

- **a)** `malloc_shared` allocates memory on the hard drive; `malloc_device` allocates memory in CPU registers
- **b)** `malloc_shared` provides memory accessible by both host CPU and accelerator device with automatic hardware/driver page migration; `malloc_device` allocates high-bandwidth memory residing strictly on the accelerator, inaccessible by host CPU pointers
- **c)** `malloc_device` is only for Intel CPUs; `malloc_shared` is only for NVIDIA GPUs
- **d)** There is no difference; they are synonymous macros in SYCL 2020

Candidate Selection: [ ]

Question 50 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations]

Question: Which exact compiler invocation compiles a SYCL C++ program `dpcpp_vector_add.cpp` into an optimized native binary using Intel oneAPI on Linux?

- **a)** `gcc -O3 dpcpp_vector_add.cpp -o vector_add`
- **b)** `icpx -fsycl -O3 dpcpp_vector_add.cpp -o vector_add`
- **c)** `python -m compile dpcpp_vector_add.cpp`
- **d)** `make -sycl dpcpp_vector_add.cpp`

Candidate Selection: [ ]
